

Betriebssystem- Entwicklung

1. Einführung

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

- Organisation
- Lernziele und Nutzen
- Inhalte
- Literatur
- Einordnung der Veranstaltung

1.1 Organisation

1.2 Lernziele und Nutzen

1.3 Inhalte

1.4 Literatur

1.5 Einordnung der Veranstaltung

- Vorlesung: 1 SWS und Übung: 3 SWS
- Präsenztermine
 - Montag, 10:30 - 10:00Uhr, 25.12.01.51
 - Donnerstag, 14:30 - 16:00Uhr, 25.12.01.51
- Hauptziel: Jede Person baut ein eigenes kleines 64-Bit Betriebssystem für x86 (Name: **HeineOS**)
 - Dies entsteht durch die Übungen
 - Für die verschiedenen Komponenten gibt es jeweils Vorgaben, sodass nicht alles selbst programmiert werden muss

1.1 Organisation

- Vorlesungs- und Übungsunterlagen:
auf GitHub (<https://github.com/hhu-bsinfo/HeineOS>)
- Rocketchat-Grupe (Betriebssystementwicklung 2026)
- Voraussetzungen:
 - Betriebssystem-Grundkenntnisse empfohlen
 - Spaß and hardwarenaher Programmierung
 - Durchhaltevermögen

- Ziel: jede Person soll am Ende ein eigenes kleines Betriebssystem geschrieben haben
→ verpflichtende Abgabe
- Die Übungsblätter bauen aufeinander auf, wodurch sich das Betriebssystem ergibt
- Das Betriebssystem muss in Qemu laufen!
 - Rechner zum Testen auf echter Hardware wird in der Übung bereitgestellt
- Die Programmierung erfolgt in Rust (punktuell wird Assembler benötigt)

- 5 ECTS für den Studiengang Master Informatik
- Prüfungsform: Portfolio + Prüfungsgespräch
 - Das eigene Betriebssystem muss zwei Wochen vor dem Gespräch abgegeben werden
 - Im Prüfungsgespräch muss eine Demo vorgeführt werden
 - Für die Note 1.0 müssen alle Funktionen aus allen Übungsblättern funktionieren sowie eine Anwendung oder Erweiterung vorzeigbar sein.
 - Die Beantwortung der Fragen zum eigenen Projekt fließt auch in die Bewertung ein.

1.1 Organisation

1.2 Lernziele und Nutzen

1.3 Inhalte

1.4 Literatur

1.5 Einordnung der Veranstaltung

1.2 Lernziele und Nutzen

- Vertieftes Verständnis für Betriebssystem-Techniken/-Konzepten
 - Struktur
 - Funktionsweise
 - Implementierung
- Grundlegende BS-Funktionen in Rust implementieren
- Der Weg ist das Ziel: **HeineOS**
 - Entwicklung eines eigenen Betriebssystems von der Pike auf
 - x86-Technologie besser verstehen

1.2 Lernziele und Nutzen

- Kenntnisse nützlich für die Entwicklung
 - von Kernel-Komponenten und Treibern
 - von Middleware und Anwendungen
- Zusätzliche Möglichkeiten auf der Hardware-Ebene
- Bei Bedarf Leistungsreserven der Hardware voll nutzbar

Programming to the bare metal

- „If you want to travel around the world and be invited to speak at a lot of different places, just write a UNIX operating system“ - Linus Torvalds

1.1 Organisation

1.2 Lernziele und Nutzen

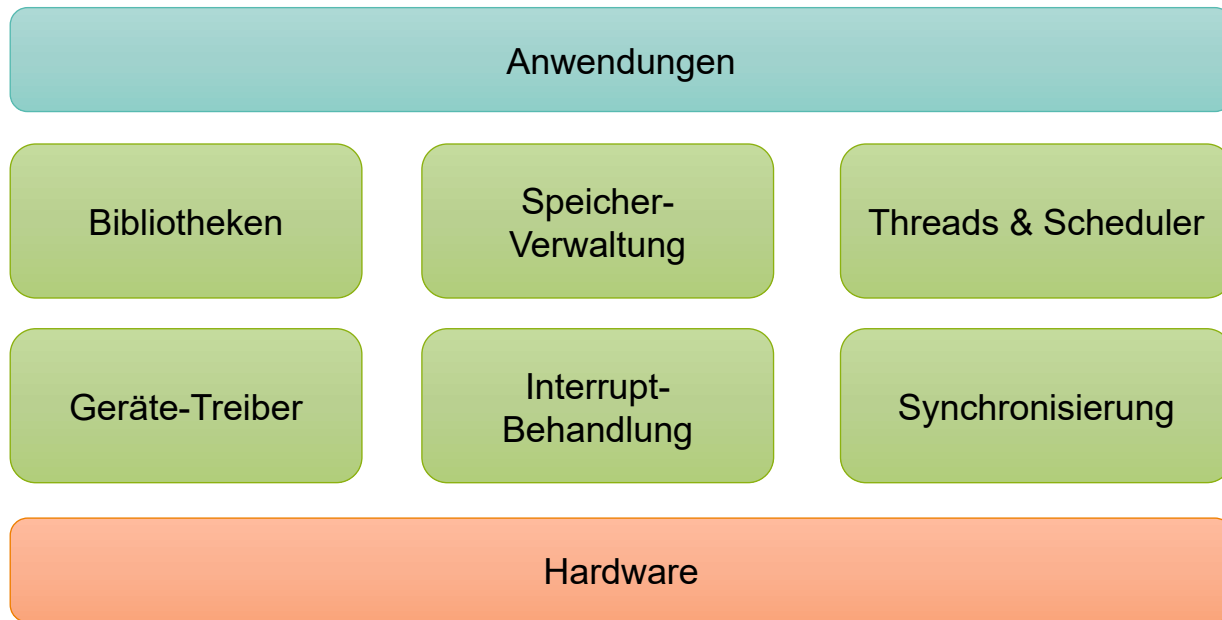
1.3 Inhalte

1.4 Literatur

1.5 Einordnung der Veranstaltung

1. Einleitung (mit PC-Speaker)
2. BS-Entwicklung (Bootvorgang & Debugging)
3. x86-64 Architektur
4. Interrupts (PIC & APIC)
5. Koroutinen & Threads & Scheduling
6. Synchronisierung
7. Gerätetreiberarchitekturen (Linux, Windows)

1. Ein-/Ausgabe: Serielle Schnittstelle, Bildschirm & Tastatur
2. Heap-Verwaltung
3. Interrupts: PIC & Tastatur
4. Nebenläufigkeit: Koroutinen, kooperative Threads, Scheduler
5. Preemptives Multi-Threading
6. Dateisystem & GameBoy Emulator (Peanut-GB)
7. Eine eigene Anwendung oder Erweiterung



1.1 Organisation

1.2 Lernziele und Nutzen

1.3 Inhalte

1.4 Literatur

1.5 Einordnung der Veranstaltung

1.4 Literatur

- <https://OSDev.org>
- Manuals von Intel und AMD

1.1 Organisation

1.2 Lernziele und Nutzen

1.3 Inhalte

1.4 Literatur

1.5 Einordnung der Veranstaltung

1.5 Einordnung der Veranstaltung

